GenericRx: Offline-First Medicine Alternative & Cost Audit System

Course / Subject: Software Engineering / Web Development / Applied Computer Science
Project Type: College Mini-Project
Domain: Healthcare Tech, Progressive Web Applications (PWA), Data Management
Backend & Core Engine: Python (FastAPI + SQLite + RapidFuzz + SQLAlchemy)
Frontend & Offline Layer: HTML5, Modern CSS Glassmorphism, Service Worker, IndexedDB (Dexie.js), Fuse.js

---

Executive Summary & Abstract

When patients visit retail pharmacies—especially near hospitals, emergency care centers, or in rural clinics—they frequently face situations where their prescribed brand-name medications are out of stock or prohibitively expensive. Most patients are unaware that equivalent, government-approved generic drugs (such as Jan Aushadhi equivalents) exist with the exact same Active Pharmaceutical Ingredient (API), dosage strength, and therapeutic effect at up to 85% lower cost.

Furthermore, hospital basements, rural clinics, and emergency pharmacy rooms often have weak or zero cellular connectivity (2G/3G or signal blind spots), rendering traditional online search applications completely unusable when needed most.

To solve this dual challenge, **GenericRx** was engineered as a lightweight, neutral, Offline-First Progressive Web Application (PWA). Powered by a Python FastAPI backend and an offline-syncing IndexedDB client storage layer, GenericRx allows users to search for any brand-name medication, resolve its active chemical composition, discover generic alternatives, audit prescription costs, flag regulatory warnings (Schedule H/X), and function 100% offline without an active internet connection.

---

1. System Architecture & Offline Cache Flow

GenericRx implements a dual-tier architecture with seamless online/offline fallback capability.

1.1 High-Level System Architecture Diagram

```
graph TD
   A[Client User Interface PWA] --> B{Network Status?}
   B -- Online --> C[Python FastAPI REST Backend]
   B -- Offline --> D[Client-Side IndexedDB / Dexie.js]

   C --> E[(SQLite Database genericrx.db)]
   C --> F[Python RapidFuzz Search Engine]
   C --> G[Python Cost Savings Engine]

   E -- Initial Sync / Service Worker --> D
   D --> H[Fuse.js Offline Search Engine]
   D --> I[JavaScript Offline Savings Calculator]

   F --> J[Search & Salt Mapping Response]
   H --> J

   G --> K[Cost Audit & Regulatory Warnings]
   I --> K
```

1.2 Offline Service Worker Cache Flow

```
sequenceDiagram
   autonumber
   actor Patient
   participant PWA as PWA Shell (HTML/CSS/JS)
   participant SW as Service Worker (sw.js)
   participant IDB as IndexedDB (GenericRxDB)
   participant API as Python FastAPI Backend

   Patient->>PWA: Open App / Search Medication
   PWA->>SW: Intercept HTTP Fetch
   alt Device is Online
       SW->>API: GET /api/search?q=Crocin
       API-->>SW: Return 567 Seeded Medicines + Alternatives
       SW-->>PWA: Render Results
       PWA->>IDB: Sync Latest Dataset for Offline Use
   else Device is Offline / Basement Mode
       SW-->>PWA: Serve Cached Static App Shell
       PWA->>IDB: Query Local Database via Fuse.js
       IDB-->>PWA: Return Cached Matches & Savings Calculation
       PWA-->>Patient: Render 100% Offline Results with "Offline Mode" Pill
   end
```

---

2. Entity-Relationship (ER) & Database Schema Design

The system relies on a clean, normalized relational model stored in SQLite (`genericrx.db`) server-side and mirrored into IndexedDB client-side.

2.1 Entity-Relationship Diagram

```
erDiagram
   MEDICINE {
       int id PK
       string name "Brand / Generic Name"
       string salt_name "Active Salt Composition"
       string strength "Dosage Strength e.g. 650mg"
       string dosage_form "Tablet / Syrup / Injection"
       float price_per_unit "Price in INR"
       int unit_count "Units per strip/bottle"
       boolean is_generic "True for Jan Aushadhi"
       string manufacturer "Company Name"
       string schedule "OTC / Schedule H / Schedule X"
       string therapeutic_category "Analgesic / Antibiotic / etc."
       text description "Clinical usage notes"
   }

   PRESCRIPTION_BASKET {
       int basket_id PK
       int medicine_id FK
       int alternative_id FK
       int monthly_units "Units prescribed per month"
   }

   MEDICINE ||--o{ PRESCRIPTION_BASKET : "prescribed in"
```

2.2 Relational Schema Field Descriptions

Field Name	Type	Key	Description
`id`	INTEGER	Primary Key	Unique identifier for medicine
`name`	VARCHAR(255)	Index	Trade/Brand or Generic name (e.g. Crocin 650)

Field Name	Type	Key	Description
`salt_name`	VARCHAR(255)	Index	Active Pharmaceutical Ingredient (e.g. Paracetamol 650mg)
`strength`	VARCHAR(100)	-	Formulation dosage strength (e.g. 650mg, 500mg+125mg)
`dosage_form`	VARCHAR(100)	-	Tablet, Capsule, Syrup, Injection, Ointment
`price_per_unit`	FLOAT	-	Price per strip or bottle in local currency (INR)
`unit_count`	INTEGER	-	Number of dosage units per container (e.g. 10 tablets)
`is_generic`	BOOLEAN	Index	Flag indicating generic/Jan Aushadhi status
`manufacturer`	VARCHAR(255)	-	Pharmaceutical manufacturer name
`schedule`	VARCHAR(50)	-	Regulatory control status (OTC, Schedule H, Schedule X)

Field Name	Type	Key	Description
`therapeutic_category`	VARCHAR(100)	Index	Drug class (Analgesic, Antibiotic, Antidiabetic, etc.)

---

3. Core Functional Modules & Mathematical Models

Module 1: Intelligent Medicine & Chemical Search

- **Brand to Salt Mapping**: Resolves trade names immediately to active salt composition (e.g., *Augmentin 625* (\rightarrow) *Amoxicillin 500mg + Clavulanic Acid 125mg*).
- **Composition Search**: Direct chemical name queries list all available brands and generic alternatives.
- **Typo-Tolerant Fuzzy Search**: Uses `rapidfuzz` (server-side Python) and `Fuse.js` (client-side offline) to handle spelling mistakes (e.g. "crocen", "dolo", "augmantin").

Module 2: Cost Audit & Savings Engine

- **Direct Savings Formula**: $$\text{Savings \%} = \left(\frac{\text{Brand Price} - \text{Generic Price}}{\text{Brand Price}} \right) \times 100$$
- **Visual Audit Badges**:
- ■ **Green Badge**: Savings (\ge 50\%) (High Cost Relief)
- ■ **Amber Badge**: Savings (20\% - 50\%) (Moderate Savings)
- ■ **Blue Badge**: Savings (< 20\%) (Low Savings)
- **Prescription Basket Accumulator**: $$\text{Monthly Savings} = \sum_{i=1}^{N} \left(P_{\text{brand}, i} - P_{\text{generic}, i} \right) \times U_i$$ $$\text{Annual Savings} = \text{Monthly Savings} \times 12$$

Module 3: Offline-First PWA Layer

- **Client-Side Database**: Pre-seeded with 567 Indian and Global medicines.
- **Service Worker (`sw.js`)**: Caches app shell static assets (HTML/CSS/JS) and API responses.
- **Network Status Listener**: Dynamic status pill ("■ Python Server Connected" vs "■ Offline Mode (IndexedDB Active)").

Module 4: Regulatory Safety & Warning Badges

- **Prescription Requirement Tags**: Highlights controlled substances requiring a doctor's prescription:
- **Schedule H / H1**: Red warning tags (e.g., Antibiotics, Benzodiazepines).
- **Schedule X**: High-risk controlled narcotics/stimulants (e.g., Methylphenidate).
- **OTC**: Blue tag for Over-The-Counter drugs.
- **Dosage Mismatch Alert**: Automatically flags alternatives with strength differences (e.g., 500mg vs 650mg) or form variations (Tablet vs Syrup).

---

4. Evaluation & Verification Checklist

Criteria	Status	Implementation Details
1. Search & Chemical Matching	PASS	Brand-to-salt resolution & RapidFuzz/Fuse.js typo tolerance.
2. Offline Capabilities (PWA)	PASS	Service Worker asset caching & Dexie IndexedDB sync.
3. Cost Audit Engine	PASS	Direct percentage savings model, visual badges, annual report export.
4. Database & Code Architecture	PASS	Python 3.14 + FastAPI + SQLite + clean modular structure.
5. Safety & Regulatory Alerts	PASS	Schedule H/X prescription badges & dosage mismatch warnings.

---

5. Execution Instructions for Faculty / Reviewers

1. **Prerequisites**: Python 3.8+ installed.
2. **Install Dependencies**: `bash pip install -r requirements.txt`
3. **Seed Database**: `bash python -m app.seed_database`

4. **Launch Application**: `bash python run.py`
5. **Access Application**:
6. Web App UI: `http://127.0.0.1:8000`
7. OpenAPI / Swagger Documentation: `http://127.0.0.1:8000/docs`

---

6. Conclusion & Future Enhancements

GenericRx successfully bridges the gap between healthcare affordability and connectivity constraints. By combining Python's high-performance FastAPI backend with an Offline-First Progressive Web Application architecture, patients can save up to 85% on their prescriptions even when standing in zero-reception hospital basements.

Future Scope: - OCR Prescription Scanning via Tesseract.js / Python OpenCV. - Multilingual Voice Search in regional languages. - Integration with National Jan Aushadhi Kendra Store Locator APIs.